LAB REPORT 7

Mobility and Multi-AP Handover in Mininet-WiFi

Student Name	Hadis Rodpradit
Student ID	6630613039
Program	Digital Engineering International Program, College of Computing, PSU Phuket
Course	977-337 Network Programming and Simulation
Week	7
Environment	Ubuntu Linux VM, Mininet-WiFi Python API (`mn_wifi`), Open vSwitch APs
Source file	`lab07_mobility/mobility_topo.py` in the netprog-2026 repository

1. Objective

This lab connects two access points with a wired backbone to form one roaming network, then moves a station (sta3) in a straight line from ap1 to ap2 using scripted mobility. I logged sta3's position and association every second to observe the handover, ran a background ping during the walk to check for interruptions, and finally reduced both APs' range so that their coverage no longer overlapped, creating a dead zone.

2. Topology and Expected Coverage

Node	Position	SSID / channel	Coverage on the x-axis
ap1	20,50,0	netprog-ap1 / ch 1 (2412 MHz)	range 35: x = −15 … **55** (range 15: x = 5 … **35**)
ap2	80,50,0	netprog-ap2 / ch 6 (2437 MHz)	range 35: x = **45** … 115 (range 15: x = **65** … 95)
sta1	15,50,0	—	fixed, on ap1
sta2	85,50,0	—	fixed, on ap2
sta3	10,50,0 → 90,50,0	—	moves from t = 1 s to t = 20 s

sta3 travels 80 m in 19 s, so its speed is 80 / 19 ≈ **4.21 m/s** and its position is x(t) = 10 + 4.21 × (t − 1). With range = 35 the two cells overlap between x = 45 m and x = 55 m. sta3 enters ap2's coverage at x = 45 m (t ≈ 9.3 s) and leaves ap1's coverage at x = 55 m (t ≈ **11.7 s**). With range = 15 there is a 30 m gap between x = 35 m (t ≈ 6.9 s) and x = 65 m (t ≈ 14.1 s).

3. Final Script and Changes Needed in This Environment

The skeleton in the lab sheet needed three changes to work on this installation of Mininet-WiFi. Each change was confirmed by a failed run first:

#	Problem observed	Cause	Fix
1	Part B pingall: **100% dropped** although sta1 and sta2 were both associated (Figure 1)	A script using `Mininet_wifi()` starts no controller, so the OVS-based APs had no flow rules and dropped every frame sent over the backbone port	`failMode='standalone'` on both APs, so OVS works as a normal learning switch

#	Problem observed	Cause	Fix
2	Every log line printed `associatedTo: None`	This version stores the association on the wireless interface, not in `sta3.params`	Read `sta3.wintfs[0].associatedTo` when `params` is empty
3	sta3 stayed on ap1 for all 22 s	Mobility declared after `net.start()` was never started, so sta3 never moved	Declare `startMobility` … `stopMobility` before `net.build()`; print sta3's x position to prove it moves

The final version below is the Part C/E configuration (overlapping `range=35` and the ping-logging line), which is the version pushed to the repository:

```
import time
from mn_wifi.net import Mininet_wifi
from mn_wifi.cli import CLI
from mininet.log import setLogLevel, info

def topology():
    net = Mininet_wifi()

    info("*** Creating nodes\n")
    sta1 = net.addStation('sta1', position='15,50,0')
    sta2 = net.addStation('sta2', position='85,50,0')
    sta3 = net.addStation('sta3', position='10,50,0')
    ap1 = net.addAccessPoint('ap1', ssid='netprog-ap1', mode='g', channel='1',
                             position='20,50,0', range=35, failMode='standalone')
    ap2 = net.addAccessPoint('ap2', ssid='netprog-ap2', mode='g', channel='6',
                             position='80,50,0', range=35, failMode='standalone')

    info("*** Configuring wifi nodes\n")
    net.configureWifiNodes()
    net.addLink(ap1, ap2)

    # Mobility must be declared BEFORE net.build() in this Mininet-WiFi version
    info("*** Configuring mobility\n")
    net.startMobility(time=0)
    # TODO 2
    net.mobility(sta3, 'start', time=1, position='10,50,0')
    # TODO 3
    net.mobility(sta3, 'stop', time=20, position='90,50,0')
    net.stopMobility(time=21)

    info("*** Starting network\n")
    net.build()
    net.start()

    # Part E: background ping from sta3 to sta1 during the walk
    sta3.cmd('ping -i 0.5 10.0.0.1 > /tmp/sta3_ping.log &')

    info("*** Monitoring sta3 association during mobility\n")
    for t in range(22):
        # TODO 4: this Mininet-WiFi version stores the association on the wireless interface
        assoc = sta3.params.get('associatedTo') or sta3.wintfs[0].associatedTo
        x = round(float(sta3.position[0]), 1)
        print(f"t={t}s sta3 x={x}m associated to: {assoc}")
        time.sleep(1)

    CLI(net)
    net.stop()

if __name__ == "__main__":
    setLogLevel("info")
    topology()
```

4. Lab Tasks and Results

Task 1 — Two APs on One Roaming Network (Part B)

```
mininet-wifi> pingall
*** Ping: testing ping reachability
sta1 -> *** sta1 : ('ping -c1  10.0.0.2',)
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

X
sta2 -> *** sta2 : ('ping -c1  10.0.0.1',)
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
From 10.0.0.2 icmp_seq=1 Destination Host Unreachable

--- 10.0.0.1 ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

X
*** Results: 100% dropped (0/2 received)
mininet-wifi>
mininet-wifi> sta1 iw dev sta1-wlan0 link
Connected to 02:00:00:00:02:00 (on sta1-wlan0)
        SSID: netprog-ap1
        freq: 2412.0
        RX: 25976 bytes (559 packets)
        TX: 1321 bytes (15 packets)
        signal: -49 dBm
        tx bitrate: 54.0 MBit/s
        bss flags: short-slot-time
        dtim period: 2
        beacon int: 100
mininet-wifi>
mininet-wifi> sta2 iw dev sta2-wlan0 link
Connected to 02:00:00:00:03:00 (on sta2-wlan0)
        SSID: netprog-ap2
        freq: 2437.0
        RX: 32190 bytes (694 packets)
        TX: 1529 bytes (18 packets)
        signal: -49 dBm
        tx bitrate: 54.0 MBit/s
        bss flags: short-slot-time
        dtim period: 2
        beacon int: 100
```

Figure 1: First attempt without failMode: both stations associated, but pingall 100% dropped

```
mininet-wifi> pingall
*** Ping: testing ping reachability
sta1 -> *** sta1 : ('ping -c1  10.0.0.2',)
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.49 ms

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.493/1.493/1.493/0.000 ms
sta2
sta2 -> *** sta2 : ('ping -c1  10.0.0.1',)
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.731 ms

--- 10.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.731/0.731/0.731/0.000 ms
sta1
*** Results: 0% dropped (2/2 received)
mininet-wifi>
mininet-wifi> sta1 iw dev sta1-wlan0 link
Connected to 02:00:00:00:02:00 (on sta1-wlan0)
        SSID: netprog-ap1
        freq: 2412.0
        RX: 164765 bytes (3554 packets)
        TX: 2025 bytes (24 packets)
        signal: -49 dBm
        rx bitrate: 54.0 MBit/s
        tx bitrate: 1.0 MBit/s
        bss flags: short-slot-time
        dtim period: 2
        beacon int: 100
mininet-wifi>
mininet-wifi> sta2 iw dev sta2-wlan0 link
Connected to 02:00:00:00:03:00 (on sta2-wlan0)
        SSID: netprog-ap2
        freq: 2437.0
        RX: 169240 bytes (3658 packets)
        TX: 2025 bytes (24 packets)
        signal: -49 dBm
        rx bitrate: 54.0 MBit/s
        tx bitrate: 1.0 MBit/s
        bss flags: short-slot-time
        dtim period: 2
        beacon int: 100
```

Figure 2: With failMode='standalone': pingall 0% dropped; sta1 on netprog-ap1, sta2 on netprog-ap2

sta1 is associated with **netprog-ap1** (BSSID …:02:00, 2412 MHz, channel 1) and sta2 with **netprog-ap2** (BSSID …:03:00, 2437 MHz, channel 6), yet pingall reports **0% dropped (2/2 received)**. Because the two stations are on different APs and different channels, the only path between them is the wired backbone `addLink(ap1, ap2)` — this proves the backbone works and the two APs form one bridged network. The first ping (1.49 ms) is slower while the AP switches learn the MAC addresses.

Task 2 — Per-Second Association Log (Part C)

```
*** Configuring wifi nodes
*** ap1-wlan1: minimum tx power (1 dBm) yields 35.00m for requested 35.00m (delta +0.00m)
*** ap2-wlan1: minimum tx power (1 dBm) yields 35.00m for requested 35.00m (delta +0.00m)
*** Configuring mobility
*** Starting network
*** Starting controller(s)
*** Starting L2 nodes
ap1 ap2 ...
*** Monitoring sta3 association during mobility
t=0s sta3 x=10.0m associated to: ap1-wlan1
t=1s sta3 x=12.5m associated to: ap1-wlan1
t=2s sta3 x=16.7m associated to: ap1-wlan1
t=3s sta3 x=20.9m associated to: ap1-wlan1
t=4s sta3 x=25.5m associated to: ap1-wlan1
t=5s sta3 x=29.7m associated to: ap1-wlan1
t=6s sta3 x=33.9m associated to: ap1-wlan1
t=7s sta3 x=38.1m associated to: ap1-wlan1
t=8s sta3 x=42.3m associated to: ap1-wlan1
t=9s sta3 x=46.5m associated to: ap1-wlan1
t=10s sta3 x=50.7m associated to: ap1-wlan1
t=11s sta3 x=54.9m associated to: ap1-wlan1
t=12s sta3 x=59.1m associated to: ap2-wlan1
t=13s sta3 x=63.8m associated to: ap2-wlan1
t=14s sta3 x=68.0m associated to: ap2-wlan1
t=15s sta3 x=72.2m associated to: ap2-wlan1
t=16s sta3 x=76.4m associated to: ap2-wlan1
t=17s sta3 x=80.6m associated to: ap2-wlan1
t=18s sta3 x=84.8m associated to: ap2-wlan1
t=19s sta3 x=89.0m associated to: ap2-wlan1
t=20s sta3 x=89.8m associated to: ap2-wlan1
t=21s sta3 x=89.8m associated to: ap2-wlan1
```

Figure 3: sta3 association log with range=35, t = 0 … 21 s

The log runs for the full 22 seconds. sta3's x position increases by ≈4.2 m every second, from 10.0 m to 89.8 m, and its association changes once, from **ap1-wlan1** to **ap2-wlan1**.

Task 3 — Handover Time Analysis (Part D)

t (s)	x (m)	Distance to ap1	Distance to ap2	Associated to
9	46.5	26.5 m (in range)	33.5 m (in range)	ap1-wlan1
10	50.7	30.7 m (in range)	29.3 m (in range)	ap1-wlan1
11	54.9	34.9 m (in range)	25.1 m (in range)	ap1-wlan1
12	**59.1**	**39.1 m (out of range)**	20.9 m (in range)	**ap2-wlan1**

The handover happened **between t = 11 s and t = 12 s**, while sta3 moved from x = 54.9 m to x = 59.1 m. This matches the calculation: sta3 leaves ap1's coverage at x = 20 + 35 = 55 m, at t = 1 + (55 − 10) / 4.21 ≈ **11.7 s**. sta3 had already been inside ap2's coverage since x = 45 m (t ≈ 9.3 s), so from t ≈ 9.3 s to 11.7 s it was in the overlap zone and could hear both APs. It nevertheless stayed on ap1 until it went out of ap1's range: the default handover behaviour keeps the current AP while it is still reachable rather than switching to the stronger one, and the overlap zone is what lets ap2 take over immediately when ap1 is lost.

Task 4 — Connectivity During the Handover (Part E)

```
mininet-wifi> sta3 cat /tmp/sta3_ping.log
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
From 10.0.0.3 icmp_seq=1 Destination Host Unreachable
From 10.0.0.3 icmp_seq=2 Destination Host Unreachable
From 10.0.0.3 icmp_seq=3 Destination Host Unreachable
From 10.0.0.3 icmp_seq=4 Destination Host Unreachable
From 10.0.0.3 icmp_seq=5 Destination Host Unreachable
From 10.0.0.3 icmp_seq=6 Destination Host Unreachable
From 10.0.0.3 icmp_seq=7 Destination Host Unreachable
64 bytes from 10.0.0.1: icmp_seq=8 ttl=64 time=1037 ms
64 bytes from 10.0.0.1: icmp_seq=9 ttl=64 time=529 ms
64 bytes from 10.0.0.1: icmp_seq=10 ttl=64 time=25.6 ms
64 bytes from 10.0.0.1: icmp_seq=11 ttl=64 time=1.09 ms
64 bytes from 10.0.0.1: icmp_seq=12 ttl=64 time=1.13 ms
64 bytes from 10.0.0.1: icmp_seq=13 ttl=64 time=1.31 ms
64 bytes from 10.0.0.1: icmp_seq=14 ttl=64 time=1.47 ms
64 bytes from 10.0.0.1: icmp_seq=15 ttl=64 time=1.53 ms
64 bytes from 10.0.0.1: icmp_seq=16 ttl=64 time=1.63 ms
64 bytes from 10.0.0.1: icmp_seq=17 ttl=64 time=1.64 ms
64 bytes from 10.0.0.1: icmp_seq=18 ttl=64 time=1.90 ms
64 bytes from 10.0.0.1: icmp_seq=19 ttl=64 time=2.02 ms
64 bytes from 10.0.0.1: icmp_seq=20 ttl=64 time=2.04 ms
64 bytes from 10.0.0.1: icmp_seq=21 ttl=64 time=2.05 ms
64 bytes from 10.0.0.1: icmp_seq=22 ttl=64 time=2.19 ms
64 bytes from 10.0.0.1: icmp_seq=23 ttl=64 time=2.34 ms
64 bytes from 10.0.0.1: icmp_seq=24 ttl=64 time=4.26 ms
64 bytes from 10.0.0.1: icmp_seq=25 ttl=64 time=1.70 ms
64 bytes from 10.0.0.1: icmp_seq=26 ttl=64 time=1.60 ms
64 bytes from 10.0.0.1: icmp_seq=27 ttl=64 time=1.70 ms
64 bytes from 10.0.0.1: icmp_seq=28 ttl=64 time=1.42 ms
64 bytes from 10.0.0.1: icmp_seq=29 ttl=64 time=1.17 ms
64 bytes from 10.0.0.1: icmp_seq=30 ttl=64 time=1.11 ms
64 bytes from 10.0.0.1: icmp_seq=31 ttl=64 time=0.975 ms
64 bytes from 10.0.0.1: icmp_seq=32 ttl=64 time=0.927 ms
64 bytes from 10.0.0.1: icmp_seq=33 ttl=64 time=0.748 ms
64 bytes from 10.0.0.1: icmp_seq=34 ttl=64 time=0.536 ms
64 bytes from 10.0.0.1: icmp_seq=35 ttl=64 time=0.506 ms
64 bytes from 10.0.0.1: icmp_seq=36 ttl=64 time=0.664 ms
64 bytes from 10.0.0.1: icmp_seq=37 ttl=64 time=0.844 ms
64 bytes from 10.0.0.1: icmp_seq=38 ttl=64 time=1.03 ms
64 bytes from 10.0.0.1: icmp_seq=39 ttl=64 time=1.08 ms
64 bytes from 10.0.0.1: icmp_seq=40 ttl=64 time=1.13 ms
64 bytes from 10.0.0.1: icmp_seq=41 ttl=64 time=1.34 ms
64 bytes from 10.0.0.1: icmp_seq=42 ttl=64 time=1.20 ms
64 bytes from 10.0.0.1: icmp_seq=43 ttl=64 time=1.14 ms
64 bytes from 10.0.0.1: icmp_seq=44 ttl=64 time=1.28 ms
64 bytes from 10.0.0.1: icmp_seq=45 ttl=64 time=1.22 ms
64 bytes from 10.0.0.1: icmp_seq=46 ttl=64 time=1.24 ms
64 bytes from 10.0.0.1: icmp_seq=47 ttl=64 time=1.20 ms
64 bytes from 10.0.0.1: icmp_seq=48 ttl=64 time=1.19 ms
```

Figure 4: sta3 → sta1 ping log (ping -i 0.5) during the walk with range=35

The ping is sent every 0.5 s, so icmp_seq n was sent ≈ (n − 1) × 0.5 s after the walk started, and the handover at t ≈ 11.5 s corresponds to **seq ≈ 24**.

icmp_seq	Time	Result	Explanation
1–7	0–3 s	Destination Host Unreachable	start-up: OVS/ARP not ready yet — before the walk, not the handover
8–10	3.5–4.5 s	1037 / 529 / 25.6 ms	replies that waited in the queue until ARP resolved
11–23	5–11 s	≈1.1–2.3 ms	normal, sta3 on ap1

icmp_seq	Time	Result	Explanation
24	**≈11.5 s**	**4.26 ms**	**small latency spike at the handover**
25–48+	12 s →	≈0.5–1.7 ms	normal, sta3 on ap2

No pings were lost during the handover — sequence numbers continue without a gap from 8 to 48 — and the only effect was a single latency spike to 4.26 ms (about 2–4× the normal RTT) at the moment sta3 moved from ap1 to ap2. The handover is almost seamless because the coverage overlaps: ap2 was already reachable when ap1 was lost, so re-association was immediate, and because both APs are on the same bridged network sta3 kept its IP address and sta1 did not need to change anything. The unreachable replies at seq 1–7 happened before sta3 started moving and are caused by start-up, not by the handover.

Task 5 — Coverage Gap with range=15 (Part F)

```
diz@diz:~/netprog-2026/lab07_mobility$ sudo python3 mobility_topo.py
*** Creating nodes
*** Configuring wifi nodes
*** ap1-wlan1: minimum tx power (1 dBm) yields 15.00m for requested 15.00m (delta +0.00m)
*** ap2-wlan1: minimum tx power (1 dBm) yields 15.00m for requested 15.00m (delta +0.00m)
*** Configuring mobility
*** Starting network
*** Starting controller(s)
*** Starting L2 nodes
ap1 ap2 ...
*** Monitoring sta3 association during mobility
t=0s sta3 x=10.0m associated to: ap1-wlan1
t=1s sta3 x=12.9m associated to: ap1-wlan1
t=2s sta3 x=17.1m associated to: ap1-wlan1
t=3s sta3 x=21.3m associated to: ap1-wlan1
t=4s sta3 x=25.5m associated to: ap1-wlan1
t=5s sta3 x=29.7m associated to: ap1-wlan1
t=6s sta3 x=33.9m associated to: ap1-wlan1
t=7s sta3 x=38.1m associated to: None
t=8s sta3 x=42.3m associated to: None
t=9s sta3 x=47.0m associated to: None
t=10s sta3 x=51.2m associated to: None
t=11s sta3 x=55.4m associated to: None
t=12s sta3 x=59.6m associated to: None
t=13s sta3 x=63.8m associated to: None
t=14s sta3 x=68.0m associated to: ap2-wlan1
t=15s sta3 x=72.2m associated to: ap2-wlan1
t=16s sta3 x=76.4m associated to: ap2-wlan1
t=17s sta3 x=80.6m associated to: ap2-wlan1
t=18s sta3 x=84.8m associated to: ap2-wlan1
t=19s sta3 x=89.0m associated to: ap2-wlan1
t=20s sta3 x=89.8m associated to: ap2-wlan1
t=21s sta3 x=89.8m associated to: ap2-wlan1
```

Figure 5: sta3 association log with range=15: no association from t = 7 s to t = 13 s

```
mininet-wifi> sta3 cat /tmp/sta3_ping.log
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
From 10.0.0.3 icmp_seq=1 Destination Host Unreachable
From 10.0.0.3 icmp_seq=2 Destination Host Unreachable
From 10.0.0.3 icmp_seq=3 Destination Host Unreachable
From 10.0.0.3 icmp_seq=4 Destination Host Unreachable
From 10.0.0.3 icmp_seq=5 Destination Host Unreachable
From 10.0.0.3 icmp_seq=6 Destination Host Unreachable
From 10.0.0.3 icmp_seq=7 Destination Host Unreachable
64 bytes from 10.0.0.1: icmp_seq=8 ttl=64 time=1061 ms
64 bytes from 10.0.0.1: icmp_seq=9 ttl=64 time=554 ms
64 bytes from 10.0.0.1: icmp_seq=10 ttl=64 time=49.9 ms
64 bytes from 10.0.0.1: icmp_seq=11 ttl=64 time=1.07 ms
64 bytes from 10.0.0.1: icmp_seq=12 ttl=64 time=1.19 ms
64 bytes from 10.0.0.1: icmp_seq=13 ttl=64 time=1.27 ms
From 10.0.0.3 icmp_seq=14 Destination Host Unreachable
From 10.0.0.3 icmp_seq=15 Destination Host Unreachable
From 10.0.0.3 icmp_seq=16 Destination Host Unreachable
From 10.0.0.3 icmp_seq=17 Destination Host Unreachable
From 10.0.0.3 icmp_seq=18 Destination Host Unreachable
From 10.0.0.3 icmp_seq=19 Destination Host Unreachable
From 10.0.0.3 icmp_seq=20 Destination Host Unreachable
From 10.0.0.3 icmp_seq=21 Destination Host Unreachable
From 10.0.0.3 icmp_seq=22 Destination Host Unreachable
From 10.0.0.3 icmp_seq=23 Destination Host Unreachable
From 10.0.0.3 icmp_seq=24 Destination Host Unreachable
From 10.0.0.3 icmp_seq=25 Destination Host Unreachable
From 10.0.0.3 icmp_seq=26 Destination Host Unreachable
From 10.0.0.3 icmp_seq=27 Destination Host Unreachable
From 10.0.0.3 icmp_seq=28 Destination Host Unreachable
From 10.0.0.3 icmp_seq=29 Destination Host Unreachable
From 10.0.0.3 icmp_seq=30 Destination Host Unreachable
From 10.0.0.3 icmp_seq=31 Destination Host Unreachable
From 10.0.0.3 icmp_seq=32 Destination Host Unreachable
From 10.0.0.3 icmp_seq=33 Destination Host Unreachable
From 10.0.0.3 icmp_seq=34 Destination Host Unreachable
64 bytes from 10.0.0.1: icmp_seq=35 ttl=64 time=1037 ms
64 bytes from 10.0.0.1: icmp_seq=36 ttl=64 time=530 ms
64 bytes from 10.0.0.1: icmp_seq=37 ttl=64 time=26.1 ms
64 bytes from 10.0.0.1: icmp_seq=38 ttl=64 time=0.923 ms
64 bytes from 10.0.0.1: icmp_seq=39 ttl=64 time=1.09 ms
64 bytes from 10.0.0.1: icmp_seq=40 ttl=64 time=1.27 ms
64 bytes from 10.0.0.1: icmp_seq=41 ttl=64 time=1.20 ms
64 bytes from 10.0.0.1: icmp_seq=42 ttl=64 time=1.15 ms
64 bytes from 10.0.0.1: icmp_seq=43 ttl=64 time=1.21 ms
64 bytes from 10.0.0.1: icmp_seq=44 ttl=64 time=1.18 ms
```

Figure 6: sta3 → sta1 ping log with range=15: 21 consecutive pings fail in the dead zone

	Part C/E (range = 35)	Part F (range = 15)
Coverage overlap	yes, x = 45–55 m	**no — 30 m gap**, x = 35–65 m
Leaves ap1	t = 11→12 s (x ≈ 55 m)	t = 6→7 s (x ≈ 34–38 m), predicted 6.9 s
Joins ap2	immediately (t = 12 s)	t = 13→14 s (x ≈ 64–68 m), predicted 14.1 s
Time with no AP	0 s	**t = 7 … 13 s (≈7 s, about 30 m)**
Ping during the move	0 lost, one 4.26 ms spike	**seq 14–34 unreachable (21 pings ≈ 10.5 s)**, then 1037 / 530 ms while ARP recovers

With the reduced range, sta3 lost ap1 at x ≈ 35 m and did not find ap2 until x ≈ 65 m, so from t = 7 s to t = 13 s it was **associated with neither AP** (None), exactly as predicted. The ping log shows the impact: 21 consecutive pings (seq 14–34) returned Destination Host Unreachable, and when traffic resumed the first replies took about 1 s because ARP had to be resolved again. Connectivity recovered a few seconds after re-association, since ARP must time out and retry first. This compares with zero lost pings in Part E.

Task 6 — Relating This to Real Roaming

A real phone or laptop does the same thing: it stays on its current Wi-Fi AP (or cell tower) until the signal becomes too weak and then re-associates with a neighbouring AP on the same network, keeping its IP address because the APs share one backbone. Part E vs. Part F shows why overlap matters: with 10 m of overlap the switch caused only a 4.26 ms spike and no loss, but with a 30 m gap the device was disconnected for about 7 s and lost 21 pings — the kind of interruption that drops a VoIP call or stalls a video stream.

5. Verification Checklist

#	Checkpoint	Evidence	✓
1	Two-AP topology (Part B) builds; pingall between sta1 and sta2 succeeds across the backbone link	Figure 2	✓
2	Part C's monitoring loop runs for the full ~22 seconds and prints a clear ap1 -> ap2 transition	Figure 3	✓
3	Handover time range reported in Task 3 is consistent with the configured speed/positions/ranges (observed t = 11–12 s, predicted 11.7 s)	Task 3 table	✓
4	Part E's ping log captured and inspected for gaps/latency around the handover	Figure 4	✓
5	Part F's reduced-range run shows a clear period of no association (t = 7–13 s)	Figures 5–6	✓
6	mobility_topo.py pushed with the overlapping-range (Part C/E) configuration as the final version	Section 3	✓

6. Reflection Questions

Q1. Describe the two ways to configure station movement in Mininet-WiFi, and explain why scripted mobility is preferable for a reproducible classroom exercise.

A **random mobility model** (`net.setMobilityModel(model='RandomWalk', ...)`, RandomWaypoint, GaussMarkov, etc.) lets nodes wander inside an area with randomly chosen directions and speeds, which is good for studying average behaviour of many users. **Scripted mobility** (`startMobility()`, `mobility(sta, 'start'/'stop', time=..., position=...)`, `stopMobility()`) gives exact start and end positions and times, and Mininet-WiFi interpolates between them. For a classroom exercise scripted mobility is better because every run produces the same path: the handover time can be calculated in advance (11.7 s) and checked against the log (11–12 s), and every student's result can be compared and graded against the same expected answer.

Q2. Why did the two APs need addLink(ap1, ap2) to form one realistic roaming network?

Without the backbone each AP would be a separate island: stations on ap1 could only talk to other stations on ap1. In Part B, sta1 (ap1) could reach sta2 (ap2) only because frames crossed the `ap1–ap2` link. During the walk this link let sta3 keep the same IP subnet and keep reaching sta1 after it moved to ap2 — the ping simply continued. This is how enterprise Wi-Fi is built: every AP is wired to the same switched LAN, so a device can roam between APs without changing its address or breaking its sessions. In this environment the APs also needed `failMode='standalone'`, because without a controller OVS does not forward frames over that link.

Q3. What real-world design principle does the coverage-gap experiment demonstrate, and why do planners avoid gaps?

It demonstrates the principle of **overlapping coverage (cell overlap)** in Wi-Fi site surveys and cellular planning: neighbouring cells are placed so their coverage overlaps — typically about 15–20% for Wi-Fi — so a moving device can find and join the next AP or tower before it loses the current one. Without overlap the device enters a dead zone. In Part F this meant ≈7 s with no association and 21 lost pings, plus extra delay to reconnect, compared with no loss when the cells overlapped. Planners avoid gaps because users experience them as dropped calls, frozen video, and failed transfers whenever they walk through that area.

7. Conclusion

After three environment-specific fixes (standalone fail mode for the APs, reading association from `wintfs[0]`, and declaring mobility before `build()`), the two-AP network worked as one roaming network. sta3 handed over from ap1 to ap2 between t = 11 and 12 s, matching the predicted 11.7 s, with no lost pings and only a 4.26 ms latency spike. Removing the overlap created a 7 s dead zone with 21 failed pings, which shows why overlapping coverage is essential for seamless roaming.